PORTFOLIOOS

OFFICIAL ARCHITECTURE SPECIFICATION

Full System Documentation & Architecture Report

An in-depth technical manual detailing the full-stack architecture, technology stack, interactive window engine, multi-layer security protocols, and state management of PortfolioOS.

CORE FRONTEND STACK

React 19 + Vite + Tailwind v4 + Zustand

CORE BACKEND STACK

Express 5 + MongoDB + Mongoose + Bcrypt

SECURITY PROTOCOL

Double Auth (PIN + 6-Digit Email OTP)

OS UI SUBSYSTEM

React-Rnd Windowing & Framer Motion

Author: PortfolioOS Engineering Team Date: July 2026 Version: 2.4.0 (Production)

1. Executive Summary & Product Vision

PortfolioOS is a revolutionary web application that reimagines traditional personal portfolios by embedding a complete desktop operating system within the web browser. Rather than navigating static Web pages, visitors interact with a responsive desktop environment equipped with draggable windows, an animated dock, top menu bar, interactive terminal CLI, customizable themes, and integrated application suites.

> 💡 **Core Architecture Philosophy**
> PortfolioOS combines **instant client-side responsiveness** via Zustand local state with **resilient background database synchronization** via MongoDB. High-privilege administrative functions are guarded by enterprise-grade double authentication (PIN + Email OTP).

Key highlights of PortfolioOS include:

- **Browser-Native Desktop Engine:** Full windowing system with z-index stacking, minimization, maximization, dragging, resizing, and window focus management powered by `react-rnd`.
- **Multi-Factor Security:** Sensitive administrative actions (PIN resets, email updates, database wipes) require 2-step verification—a static Bcrypt-hashed PIN followed by a single-use 6-digit email OTP.
- **Interactive Terminal Subsystem:** A custom Unix-like terminal app supporting real-time commands (`help` , `skills` , `projects` , `sudo` , `matrix` , `theme` , `clear`).
- **Dynamic Theme Engine:** Live modification of CSS root variables (fonts, accent colors, wallpapers, density, animations) persisted in browser local storage.

2. Technology Stack Breakdown

2.1 Backend Technology Stack

Technology	Version	Role & Purpose in PortfolioOS
Node.js	v23.10.0	Asynchronous server runtime executing backend operations and network handlers.
Express.js	v5.2.1	REST API framework providing route controllers, session validation, CORS, and security middleware.
MongoDB & Mongoose	v9.6.2	NoSQL database and Object Data Modeling (ODM) layer for `User`, `UserConfig`, and `Otp` schemas.
Bcrypt.js	v3.0.3	10-round salted password-hashing library securing administrative PINs and OTP validation hashes.
Nodemailer	v9.0.3	Email dispatch service supporting production SMTP transports and automated Ethereal Email test accounts.
CORS	v2.8.6	Cross-Origin Resource Sharing security layer restricting API access to dynamic whitelist origins.

2.2 Frontend Technology Stack

Technology	Version	Role & Purpose in PortfolioOS
React	v19.2.7	Component UI library rendering the desktop shell, window applications, dock, and modals.
Vite	v8.1.1	Lightning-fast frontend build tool and hot-reloading dev server.
Tailwind CSS	v4.3.2	Utility-first CSS styling engine providing design system tokens and glassmorphism UI styles.
Zustand	v5.0.14	Centralized state manager controlling window stacks, active apps, themes, user preferences, and REST sync.
Framer Motion	v12.42.2	Animation library powering dock magnification, modal transitions, desktop booting animations, and window effects.
React-Rnd	v10.5.3	Resizable and draggable window container layer powering the OS desktop environment.

Technology	Version	Role & Purpose in PortfolioOS
Lucide & React Icons	v1.23 / v5.7	Vector icon systems powering window controls, taskbar indicators, and app launchers.

3. System Architecture & Flow Diagrams

FIGURE 3.1: HIGH-LEVEL PORTFOLIOOS SYSTEM ARCHITECTURE

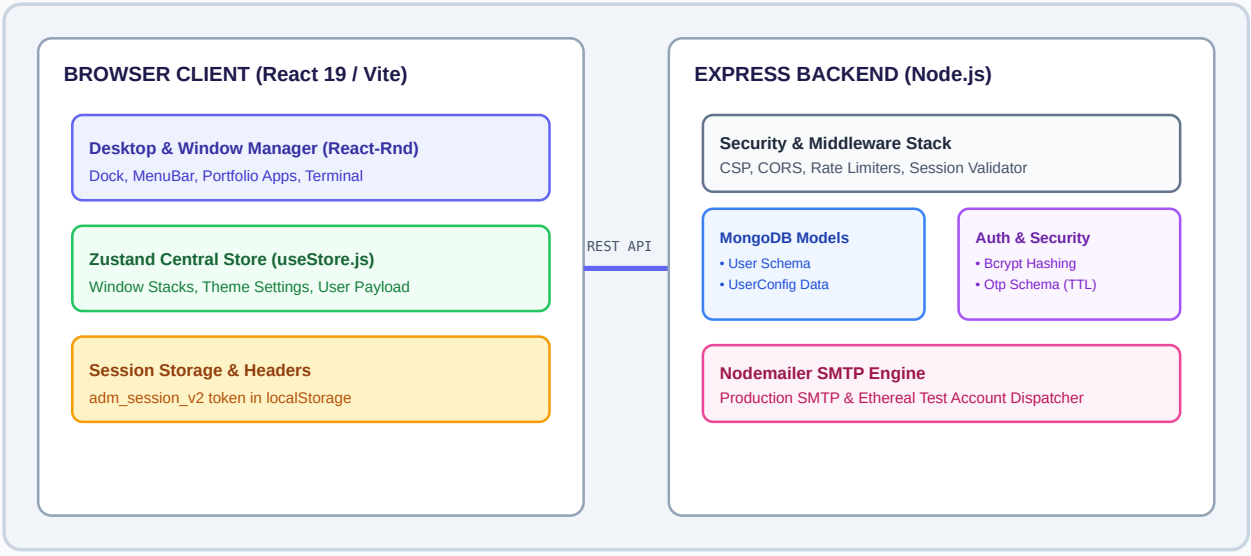
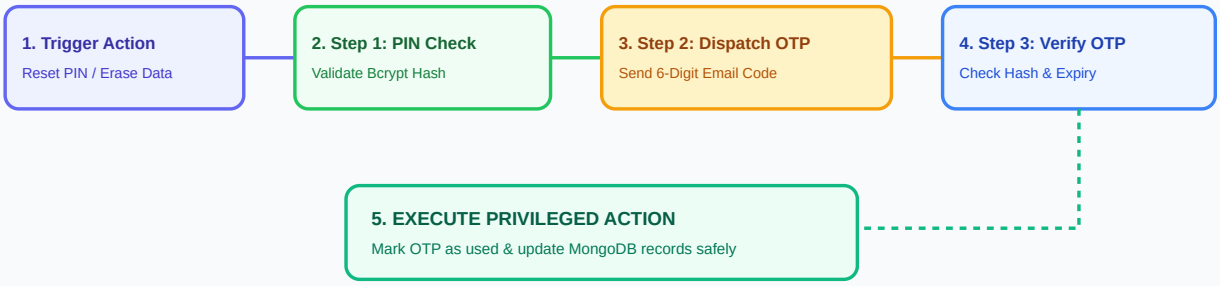


FIGURE 3.2: DOUBLE AUTHENTICATION SECURITY FLOWCHART



4. Detailed Functionality & Backend Controllers

4.1 Security & Session Functions (`server.js`)

> 🔒 **Brute-Force Rate Limiting Algorithm**
>
> The backend tracks failed auth attempts in a dynamic memory map keyed by client IP. After 10 consecutive failed attempts, further requests are blocked for 120 seconds.

```
// Rate limiting middleware for authentication attempts
function rateLimitAuth(req, res, next) {
    const ip = req.ip || req.socket?.remoteAddress || 'unknown';
    const now = Date.now();
    const record = loginAttempts.get(ip) || { count: 0, lockedUntil: 0 };

    if (record.lockedUntil > 0 && record.lockedUntil <= now) {
        loginAttempts.delete(ip);
        return next();
    }

    if (record.lockedUntil > now) {
        const waitSec = Math.ceil((record.lockedUntil - now) / 1000);
        return res.status(429).json({
            success: false,
            lockedOut: true,
            waitSeconds: waitSec,
            error: `Too many failed attempts. Try again in ${waitSec}s.`
        });
    }

    req._authIP = ip;
    next();
}
```

4.2 Comprehensive REST API Endpoint Reference

HTTP Method	Endpoint	Protection	Function & Description
`GET`	`/api/user`	Public	Fetches active portfolio payload, excluding sensitive hashes, and prunes expired certificates.
`POST`	`/api/auth/login`	Rate Limited	Validates admin/user PIN against Bcrypt hash and issues 1-hour session token.

HTTP Method	Endpoint	Protection	Function & Description
`POST`	`/api/auth/send-otp`	Rate Limited	Generates 6-digit random code, hashes it into `Otp` collection, and dispatches email.
`POST`	`/api/auth/reset-pin`	Rate Limited	Verifies OTP hash for reset purpose and updates salted PIN hash in MongoDB.
`POST`	`/api/user`	Session Protected	Saves updated portfolio config JSON (projects, skills, education) to database.
`POST`	`/api/profile/confirm-change-email`	Session Protected	Verifies OTP sent to new email address and updates profile `mailId` .
`DELETE`	`/api/profile/erase-data`	Session Protected	Verifies Access PIN and clears portfolio config payload completely.

5. Frontend Architecture & Windowing Engine

5.1 Zustand Central Store Architecture (`useStore.js`)

The central store orchestrates all desktop interactions, window z-index layering, modal states, and backend sync operations.

```
// Window Management in Zustand Store
openWindow: (winId, title) => set((state) => {
    const existing = state.windows.find(w => w.id === winId);
    if (existing) {
        return {
            windows: state.windows.map(w => w.id === winId ? { ...w, isMinimized: false, zIndex: 
            activeWindow: winId,
        };
    }
    return {
        windows: [...state.windows, { id: winId, title, zIndex: Date.now(), isMinimized: false, i
        activeWindow: winId,
    };
}),

closeWindow: (winId) => set((state) => ({
    windows: state.windows.filter(w => w.id !== winId),
    activeWindow: state.activeWindow === winId ? null : state.activeWindow,
})),
```

5.2 Core Desktop Subsystems

- **`Window.jsx` Subsystem:** Uses `react-rnd` to provide 60 FPS window dragging and resizing. Manages minimize, maximize, and active z-index focus styling.
- **`Dock.jsx` Subsystem:** macOS-inspired animated dock featuring Framer Motion spring physics on mouse hover, glowing active indicators, and click launchers.
- **`TerminalApp.jsx` Subsystem:** Embedded Unix-style terminal CLI parsing input commands dynamically and executing internal handlers (e.g., `sudo`, `matrix` mode, `skills` list).
- **`AdminApp.jsx` Subsystem:** Complete administrative interface for updating portfolio content, configuring bio details, managing skill categories, uploading project media, and controlling account security.

6. Advantages & Security Hardening Summary

6.1 Key Architectural Advantages

1. **Distinguished User Experience:** Offers visitors an unforgettable OS-like interface instead of traditional linear portfolio pages.
2. **Zero-Latency Local Feedback:** All UI updates apply instantly in client state and save to MongoDB asynchronously in the background.
3. **Fault-Tolerant Offline Fallback:** If database connection drops, PortfolioOS falls back gracefully to in-memory caching without breaking user session flow.
4. **Modular Component Architecture:** Every OS app (Terminal, Bento, Projects, Settings, Admin) is encapsulated in reusable React 19 components.

6.2 Multi-Layer Security Safeguards

> 🛡️ **Defense-in-Depth Security Matrix**
>
> PortfolioOS employs multi-tiered defenses across transport, authentication, database, and rate-limiting layers.

Security Layer	Threat Standard	Mitigation Technique
Authentication Factor 1	Credential Guessing	Salted Bcrypt hashing (10 rounds). Plaintext PINs are never stored or logged.
Authentication Factor 2	Session Compromise	Double Authentication requiring 6-digit email OTP for sensitive operations.
IP Rate Limiting	Brute Force Attack	10 failed authentication attempts trigger automatic 120-second IP lockout.
OTP Expiration & Cleanup	Replay Attack	10-minute validity duration with automatic MongoDB TTL index pruning.
HTTP Security Headers	XSS / Clickjacking / Sniffing	Strict CSP rules, X-Frame-Options DENY, X-Content-Type-Options nosniff headers.

— End of PortfolioOS Technical Documentation & Architecture Manual —